

Image Classification of the flowers-102 Dataset

Y3921738

Abstract—This report explains my process of developing a model for accurately classifying the Oxford flowers-102 dataset [1]. The task involved designing and implementing a convolutional neural network using Python [2] and the PyTorch library [3]. I conclude the report by discussing further improvements I could make including transfer learning and segmentation of the input images. I based my initial architecture on previously successful networks for image classification before modifying them to fit this specific task. Successful variants of these networks were then fine-tuned through various adjustments to the network structure and hyper parameters, starting with small-scale tests to optimize these settings on the provided dataset. Extensive testing and validation were conducted to ensure optimal performance before scaling the model up to try and achieve maximum accuracy on the dataset. The final model achieved a classification accuracy of 50.0% on the flowers-102 test set.

I. INTRODUCTION

Image classification is a fundamental task in computer vision that involves the assignment of images to a class from a predefined set. One approach to this could be flattening an image and feeding that as an input to a set of fully connected layers. However, this is inefficient for image classification as we are interested in the spacial correlation of pixels rather than individual pixel values: we need to consider not just the pixel we are looking at but also the surrounding pixels. This problem was solved in 2012 by AlexNet, a classifier designed to solve Image-Net LSVRC [4]. AlexNet was able to consider the surroundings of each pixel using convolutional layers: essentially a layer that applies multiple filters to an image to extract features, which are then stored on feature maps. This is the basis of a Convolutional Neural Network (CNN). With this, AlexNet managed to achieve an error rate of around 15%, a decrease of roughly 10% from the previous year. By 2017, 29 of the 38 teams attempting the challenge got under a 5% error rate, showing a dramatic increase in quality of CNNs over just 5 years [5]. A more recent application of image classification, "Deep Learning for Medical Image-Based Cancer Diagnosis" by X. Jiang et al. [6] discusses the application of deep neural networks on medical imagery for cancer diagnoses and their potential to be faster and more accurate than the human eye. The application can be extended into many other fields of medical imagery such as skin disease/lesion classification or pathological chest X-ray classification [7], helping medical professionals provide a higher quality of care. Based on this evidence, it appears that the most effective method for my classification problem is to use a CNN.

II. METHOD

For my image classification task, I have followed a similar structure to the AlexNet architecture[ref], but adapted the structure of the network to suit my needs. The structure of

the network can be seen in Fig. 1. The network can be split into 2 sections: feature extraction and classification.

Feature extraction is split into 3 blocks. The first block is structured as a convolutional layer generating 64 feature maps and a max-pooling layer. The second block is identical in structure but produces 128 feature maps. Finally the last block is slightly different - it contains 2 convolutional layers both producing 256 feature maps. Application of multiple convolutional layers in a network allows the model to look at groups of features on a larger scale and build an idea of the image it is looking at. Each one of the convolutional layers is followed by a batch normalisation function and a ReLU function, and each max-pooling layer is followed by a dropout layer. Batch normalisation normalises the output of the previous layer by subtracting the batch mean and dividing by the batch standard deviation. The Rectified Linear Unit (ReLU) is an activation function defined as $f(x) = \max(0, x)$. It is a very popular activation function for deep learning tasks as it is simple to compute and leads to faster convergence during training when compared to previously favoured activation functions like sigmoid or tanh. A dropout layer gives each neuron a probability (in my case 0.25) to temporarily deactivate a neuron, which helps prevent over-fitting by encouraging the network to learn more robust features.

For classification, the output of the feature extraction is flattened and then passed into a hidden layer containing 256 nodes and then to the output layer, which had 102 nodes corresponding to the 102 classes of flowers. This structure is not just different to AlexNet, but to most of the architectures I have looked at, which typically contain at least 2 hidden layers. I found these structures to be too complex for this task as we are restricted to only the training dataset of 1020 images, which is comparatively small when compared to datasets used for models like AlexNet (ILSVRC contains 1.2 million images). Using a smaller, less complex structure stops the network from over-fitting to the training data and is also computationally less expensive. The hidden layer also has a batch normalisation and a ReLU function following, but the final layer does not. The output layer could be followed by a softmax function but this isn't necessary thanks to my loss function (Cross Entropy Loss).

Cross Entropy Loss was used by AlexNet and suggested in the PyTorch tutorials [9]. It is a widely used loss function that combines Log Softmax and Negative Log Likelihood Loss. Softmax will take the raw output of our network and produce a probability distribution and loss for each example. The formula for Cross Entropy Loss for a single instance of a single class is Cross Entropy Loss = $-y_{i,c} \log(p_{i,c})$. Where $y_{i,c}$ is the label for instance i on class c where 1 represents that i is a member of c and 0 indicates it's not. $p_{i,c}$ is the model's predicted probability that i belongs to c . We can expand this out to

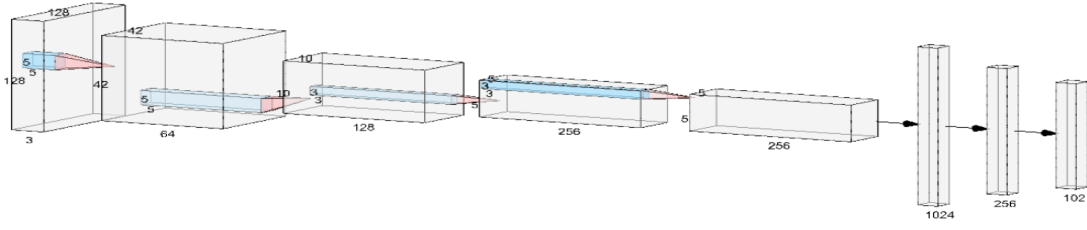


Fig. 1. Network Architecture

get the average loss function for the whole network for every instance:

$$\text{Cross Entropy Loss} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(p_{i,c})$$

Where N is the total number of instances in the data set and C is the total number of classes.

III. RESULTS & EVALUATION

To ensure the best performance of my model I needed to find the best hyper parameters, optimisation algorithm and anything else I need.

I felt the best way to go about this would be to consider the effects that changes have on a smaller scale. I limited each model to 10 epochs and a basic network so that I was able to run tests faster with my laptop. For each epoch I would calculate the loss on the training data and the validation data to use as my performance metrics, as these are good indicators of whether the model was over or under fitting. I also calculated the model's accuracy on the validation data every epoch so that I could understand what these values meant in terms of the overall accuracy of the network. With these small scale tests I could get an understanding of how my hyper-parameters affected the learning without having to wait multiple hours for the model to train every test. From here I could apply what I've found to large scale tests, which give a more accurate representation of what effect each change to a parameter will have on my network. For an optimiser, to begin with I used Adam as it was suggested by "Training a Classifier" [10] as the model would converge faster which I thought meant it was the better choice. I continued to test with both throughout the project found that despite its slower convergence, SGD resulted in a better final model, so I continued with that. I After some iterations I found that for SGD using a momentum of 0.9 and a weight decay of $5e-6$ was effective, but I didn't have enough time to ensure these were optimal. After I had implemented my optimisers, I started fine-tuning my hyper parameters by adjusting the learning rate and batch size. I found that batch size too high a batch size lead to worse generalisation, but more stable learning so based on the recommended values found in the PyTorch tutorials, I decided to use batch size 32. The optimal learning rate tended to vary based on batch size, but I found that for this a value of $1e-3$ was effective. I chose to run it for around 30 epochs as I found that just short of this point the model stopped making progress. I decided to use a learning rate scheduler, to hopefully help the model converge better even after this point, but due to time

constraints was unable to make significant progress in this area. To help decrease over-fitting, I implemented a number of data augments including random rotations, colour jitter, perspective shifts, to greater represent the natural variations of flowers. This increased the training data by 21x and was a very effective method to increase generalisation. Overall my model took 180 minutes to train on a NVIDIA GeForce GTX 1650 in a python venv environment and achieved an accuracy of 50.0% on the testing data. This was calculated by: $\frac{\text{Number of correct guesses}}{\text{Number of samples in the testing data (6149)}} * 100$

IV. CONCLUSION & FURTHER WORK

Considering the small data set and and large similarity between classes, I think my model has achieved a reasonable accuracy. I would like to have seen an accuracy above 60%, but unfortunately I was unable to achieve this in the time given. I think my architecture was a good choice as it minimised the tendency of the model to over-fit, but in later epochs both the training and validation loss stagnated indicating the model may be under-fitting.

For further work I could look into transfer learning: the process of taking a pretrained model fine-tuned for one task and applying it to a new related task. Using a pretrained model (such as vgg16) supplied by the PyTorch library, I attempt to classify the flowers-102 dataset to a greater degree of accuracy. Transfer learning would help me work past the issues that have arisen due to the small dataset as the pretrained network is unlikely to over-fit to our data.

Another of method of improvement I have looked at was the implementation of a Fully Convolutional Network (FCN) for semantic segmentation of the input images into flower and background as suggested by M. Nilsback and A. Zisserman [11]. The backgrounds of the images are quite varied as seen in Fig. 2.



Fig. 2. Flower Segmentation Example

This could cause the model to be learning nonsense from the background, which we don't want, however with segmentation shown we could potentially eliminate this distraction of the network. Unfortunately due to time constraints, I was unable to include segmentation in my model. I felt that the process for achieving accurate segmentation would take too much time out of the development of my classifier and so it was dropped in favour of focusing on improving the CNN itself.

REFERENCES

- [1] Nilsback and Zisserman. "flowers-102" Dec. 2008
- [2] Python Software Foundation. (2023, May). Python Language Reference, version 3.11 [Online]. Available: <https://www.python.org/>
- [3] PyTorch Foundation. (2023, May). PyTorch [Online]. Available: <https://pytorch.org/>
- [4] O. Russakovsky et al. (2015). ImageNet Large Scale Recognition Challenge [Online]. Available: <https://image-net.org/challenges/LSVRC/>
- [5] M. Mayank (2017, Aug). "Must-Read Path-Breaking Papers About Image Classification" [Online]. Available: <https://www.linkedin.com/pulse/must-read-path-breaking-papers-image-classification-muktabh-mayank>
- [6] X. Jiang et al. (2023, Jul). "Deep Learning for Medical Image-Based Cancer Diagnosis" [Online]. Available: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC10377683/>
- [7] H. Liao, S. Kevin Zhou and J. Luo (2023) "Deep Network Design for Medical Image Computing" Chapter 3.1.1 [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/B9780128243831000113>
- [8] S. Bangar (2022, June). "AlexNet Architecture Explained" [Online] Available: <https://medium.com/@siddheshb008/alexnet-architecture-explained-b6240c528bd5>
- [9] PyTorch Foundation. "Optimizing Model Parameters"[Online]. Available: https://pytorch.org/tutorials/beginner/basics/optimization_tutorial.html
- [10] PyTorch Foundation. "Training a classifier"[Online]. Available: https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html
- [11] M. Nilsback and A. Zisserman, "Automated flower classification over a large number of classes", Visual Geometry Group, Department of Engineering Science University of Oxford, United Kingdom. 2008 Accessed: 18/05/24 [Online]. Available: www.robots.ox.ac.uk/vgg/publications/2008/Nilsback08/nilsback08.pdf